

Supabase + Stripe + Languages Reference

Everything in one place: what this project is, every Supabase table & storage bucket, the full Stripe UPI QR donation integration, and the English/Hindi/Punjabi language feature — commands, code, and the bugs we actually hit.

Contents

[Project overview](#)

Supabase

[hero_banners](#)

[about_section](#)

[portfolio_projects](#)

[faqs](#)

[testimonials](#)

[events / events_section](#)

[gallery_items](#)

[audio_tracks / audio_section](#)

[video_section](#)

[contact_submissions](#)

[donate_section / donations](#)

[donation_tiers \(legacy\)](#)

[site_settings](#)

[profiles](#)

[Storage buckets](#)

[RLS pattern](#)

[Quick REST queries](#)

Stripe / Donate flow

[How the flow works](#)

[Get API keys](#)

[Creating the UPI QR](#)

[Registering the webhook](#)

[Verifying the webhook](#)

Languages (i18n)

[How the flow works](#)

[Static text \(next-intl\)](#)

[CMS content auto-translate](#)

[SQL migration](#)

[Language switcher](#)

[Adding a new language](#)

Ops

[Environment variables](#)

[Stripe CLI — local testing](#)

[Vercel — env vars & deploy](#)

[Gotchas we actually hit](#)

[Test mode → live mode](#)

Project overview

This is a Next.js 16 (App Router) + Supabase marketing/CMS site, deployed on Vercel. A public-facing single-page marketing site (hero, about, portfolio, testimonials, FAQs, events, gallery, audio, video, donate, contact, newsletter) is entirely driven by content the admin manages through a separate `/admin` dashboard — nothing on the public site is hardcoded content, aside from UI chrome (nav labels, buttons, empty-states).

Stack: Next.js 16.2.10 (App Router, Server Components + Server Actions), React 19.2.4, Supabase (Postgres + Auth + Storage, via `@supabase/ssr`), Tailwind CSS, GSAP for animation, Stripe for payments,

next-intl for static-text internationalization, and a free unofficial Google Translate wrapper for auto-translating CMS content.

Two halves of the app:

- **Public site** (`app/page.tsx` , `app/gallery` , `app/portfolio`) — Server Components that fetch content via `lib/supabase/server.ts` 's `get*()` functions, rendered by client components under `components/shadn/*` (each one animates in with GSAP on scroll).
- **Admin panel** (`app/admin/*`) — gated by Supabase Auth; every page checks `user.app_metadata.role === "admin"` . Each content type has a form under `components/dashboard/*-form.tsx` that writes directly to Supabase from the browser (using the logged-in admin's session, respecting RLS) and then calls `router.refresh()` so the Server Components re-fetch fresh data.

Key integrations covered in this document:

- **Supabase** — the entire content database, file storage, and auth/role gating.
- **Stripe** — a UPI QR-code donation flow (not checkout links or subscriptions — a single fixed-amount PaymentIntent shown as a scannable QR).
- **Languages (i18n)** — the site is available in English, Hindi, and Punjabi. Static UI text is translated via message files; admin-entered CMS content is auto-translated once at save time and cached in the database, so page loads never call a translation API live.

No Supabase Edge Functions or Vercel Edge Runtime are used anywhere in this project. All server-side logic — the Stripe webhook, the donation QR creation, and the CMS auto-translate pipeline — runs as ordinary Next.js Route Handlers (`app/api/*/route.ts`) and Server Actions (`app/actions/*.ts`) on the default Node.js runtime (Fluid Compute). If you came here looking for a Supabase Edge Function, there isn't one — the equivalent logic lives in the Server Action described in the Languages section below.

Supabase — all tables & buckets

This project never used Supabase CLI migrations — every table was created ad hoc via the SQL editor over time. Everything below reflects the **current live schema**. The `create table if not exists` statements below will safely reproduce this exact schema if you ever need to recreate it elsewhere. Every content table now also carries a `translations jsonb` column, added for the multi-language feature — see the [Languages section](#) for what it's for.

Naming pattern used across this project

- **Singleton "section" tables** — exactly one row, always `id = 1` , holds the copy/settings for one homepage section (e.g. `about_section` , `donate_section` , `site_settings`). Admin forms always `.update(...).eq("id", 1)` , never insert.

- **Collection tables** — many rows, uuid primary key via `gen_random_uuid()`, usually with `is_active` + `sort_order` for admin-controlled visibility/ordering (e.g. `faqs`, `testimonials`, `events`, `portfolio_projects`).

hero_banners

collection

```
create table if not exists public.hero_banners (  
  id uuid primary key default gen_random_uuid(),  
  slug text not null default 'home-hero',  
  image_url text not null,  
  image_alt text,  
  badge_emoji text default '🚀',  
  badge_text text not null,  
  title text not null,  
  description text,  
  primary_button_text text,  
  primary_button_link text,  
  secondary_button_text text,  
  secondary_button_link text,  
  stats jsonb,  
  is_active boolean not null default true,  
  sort_order integer not null default 0,  
  translations jsonb not null default '{}'::jsonb, -- hi/pa translations, see Languages section  
  created_at timestamptz not null default now(),  
  updated_at timestamptz not null default now()  
);
```

Bucket: hero-banners

about_section

singleton

```
create table if not exists public.about_section (  
  id integer primary key,  
  image_url text,  
  image_url_2 text,  
  eyebrow_text text not null default 'About Us',  
  title text not null default 'Building Digital Experiences That Inspire',  
  description text not null,  
  years_experience integer not null default 10,  
  button_text text not null default 'Learn More',  
  features text[] not null,  
  translations jsonb not null default '{}'::jsonb,  
  updated_at timestamptz not null default now()  
);  
insert into public.about_section (id) values (1) on conflict (id) do nothing;
```

Bucket: about-section

portfolio_projects

collection

```
create table if not exists public.portfolio_projects (  
  id uuid primary key default gen_random_uuid(),  
  title text not null,  
  category text not null,  
  image_url text not null,  
  project_link text,  
  description text,  
  client_name text,  
  project_year text,  
  role text,  
  is_active boolean not null default true,  
  sort_order integer not null default 0,  
  translations jsonb not null default '{}'::jsonb,  
  created_at timestamptz not null default now(),  
  updated_at timestamptz not null default now()  
);
```

Bucket: portfolio-projects

faqs

collection

```
create table if not exists public.faqs (  
  id uuid primary key default gen_random_uuid(),  
  question text not null,  
  answer text not null,  
  is_active boolean not null default true,  
  sort_order integer not null default 0,  
  translations jsonb not null default '{}'::jsonb,  
  created_at timestamptz not null default now()  
);
```

No storage bucket (text-only content).

testimonials

collection

```
create table if not exists public.testimonials (  
  id uuid primary key default gen_random_uuid(),  
  name text not null,  
  role text not null,  
  image_url text,  
  review text not null,  
  rating smallint not null default 5,  
  is_active boolean not null default true,  
  sort_order integer not null default 0,  
  translations jsonb not null default '{}':::jsonb, -- only "review" is translated; name/role are  
proper nouns  
  created_at timestamptz not null default now(),  
  updated_at timestamptz not null default now()  
);
```

Bucket: testimonials

events

collection

```
create table if not exists public.events (  
  id uuid primary key default gen_random_uuid(),  
  title text not null,  
  description text not null,  
  event_date date not null,  
  event_time text,  
  location text,  
  link_url text,  
  image_url text,  
  is_active boolean not null default true,  
  sort_order integer not null default 0,  
  translations jsonb not null default '{}':::jsonb,  
  created_at timestamptz not null default now()  
);
```

Bucket: events

events_section

singleton

```
create table if not exists public.events_section (  
  id integer primary key,  
  background_image_url text  
);  
insert into public.events_section (id) values (1) on conflict (id) do nothing;
```

Bucket: `events-section` (background image only — separate from the `events` bucket used for individual event images)

gallery_items collection

```
create table if not exists public.gallery_items (  
  id uuid primary key default gen_random_uuid(),  
  media_type text not null, -- 'image' | 'video'  
  image_url text,  
  video_url text,  
  title text,  
  is_active boolean not null default true,  
  sort_order integer not null default 0,  
  translations jsonb not null default '{}':::jsonb, -- only "title" (caption) is translatable  
  created_at timestamptz not null default now()  
);
```

Bucket: `gallery`

audio_tracks collection

```
create table if not exists public.audio_tracks (  
  id uuid primary key default gen_random_uuid(),  
  title text not null,  
  description text,  
  audio_url text not null,  
  cover_image_url text,  
  is_active boolean not null default true,  
  sort_order integer not null default 0,  
  translations jsonb not null default '{}':::jsonb,  
  created_at timestamptz not null default now()  
);
```

audio_section singleton

```
create table if not exists public.audio_section (  
  id integer primary key,  
  background_image_url text  
);  
insert into public.audio_section (id) values (1) on conflict (id) do nothing;
```

Bucket: `audio` (audio files + cover images), `audio-section` (section background)

video_section

singleton

```
create table if not exists public.video_section (  
  id integer primary key,  
  video_url text not null default '',  
  title text not null default 'See How We Bring Ideas To Life',  
  description text not null default 'A quick look at how our team designs, builds, and ships  
products our clients love.',  
  is_active boolean not null default true,  
  translations jsonb not null default '{}'::jsonb,  
  updated_at timestampz not null default now()  
);  
insert into public.video_section (id) values (1) on conflict (id) do nothing;
```

No storage bucket — `video_url` stores an external link (e.g. YouTube), not an uploaded file.

contact_submissions

collection

```
create table if not exists public.contact_submissions (  
  id uuid primary key default gen_random_uuid(),  
  full_name text not null,  
  email text not null,  
  company text,  
  phone text not null,  
  message text not null,  
  created_at timestampz not null default now()  
);
```

Bucket: `contact-section` (section background image only — submissions themselves have no file uploads). Not translated — this is visitor-submitted data, not admin-authored CMS copy.

donate_section

singleton

```
create table if not exists public.donate_section (  
  id integer primary key,  
  background_image_url text,  
  title text not null default 'Support Our Mission',  
  subtitle text not null default 'Give Back Today',  
  description text,  
  button_text text not null default 'Donate Now',  
  phone_number text,          -- "Settlement UPI ID (reference only)" field  
  razorpay_qr_code_id text,    -- legacy, unused since the Stripe switch  
  qr_code_image_url text,      -- legacy, unused since the Stripe switch  
  default_amount numeric not null default 500,  
  translations jsonb not null default '{}'::jsonb  
);  
insert into public.donate_section (id) values (1) on conflict (id) do nothing;
```

Bucket: donate-section

donations collection

```
create table if not exists public.donations (  
  id uuid primary key default gen_random_uuid(),  
  razorpay_order_id text,           -- legacy, unused since the Stripe switch  
  razorpay_payment_id text unique,  -- legacy, unused since the Stripe switch  
  stripe_payment_intent_id text unique,  
  amount numeric not null,  
  currency text not null default 'INR',  
  status text not null default 'created',  
  donor_name text,  
  donor_email text,  
  donor_contact text,  
  created_at timestamptz not null default now()  
);  
  
alter table public.donations enable row level security;  
  
drop policy if exists "Admins can read donations" on public.donations;  
create policy "Admins can read donations"  
  on public.donations for select  
  using ((auth.jwt() -> 'app_metadata' ->> 'role') = 'admin');
```

Written only by the Stripe webhook via the service-role key (bypasses RLS) — no client-side insert policy needed. No storage bucket. See the Stripe section below for how this table gets populated.

donation_tiers legacy / unused

This table exists in the database but is **not referenced anywhere in the current codebase** — left over from an earlier donation-page design (fixed donation tiers/amounts) before the QR-based Stripe flow. Safe to drop if you've confirmed nothing external depends on it.


```
-- Reconstructed schema, for reference only:
create table if not exists public.donation_tiers (
  id uuid primary key default gen_random_uuid(),
  slug text not null,
  title text not null,
  amount text not null,
  description text,
  impact text[] not null,
  is_featured boolean not null default false,
  button_text text,
  donate_url text,
  sort_order integer not null default 0,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);
```

site_settings

singleton

```
create table if not exists public.site_settings (
  id integer primary key,
  contact_phone text,
  contact_email text,
  office_address text,
  working_hours text,
  facebook_url text,
  instagram_url text,
  twitter_url text,
  linkedin_url text,
  contact_background_image_url text,
  show_login_button boolean not null default true,
  updated_at timestamptz default now()
);
insert into public.site_settings (id) values (1) on conflict (id) do nothing;
```

Used for the dashboard's "show/hide public Login button" toggle and the Contact section's details. Not translated — contact info (phone/email/address) shouldn't be machine-translated.

profiles

auth

```
create table if not exists public.profiles (
  id uuid primary key references auth.users(id) on delete cascade,
  email text not null,
  full_name text,
  role text not null default 'user', -- 'user' | 'admin'
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);
```

The actual admin gate used everywhere (route middleware, and every RLS policy in this project) reads `auth.jwt() -> 'app_metadata' -> 'role'` — not this table's `role` column directly. `app_metadata` lives on the Supabase Auth user record itself (settable only via the Admin API / dashboard, never by the user), so `profiles.role` is a readable mirror for the UI, while `app_metadata.role` is the actual source of truth enforced by every policy and by the middleware's route protection. The `profiles` table also holds `avatar_url` via `auth.users.user_metadata` for the admin's profile photo (managed through Supabase Auth's own metadata, not a separate column).

Storage buckets — all 11, all public

hero-banners about-section portfolio-projects testimonials events
events-section gallery audio audio-section donate-section contact-section
avatars (admin profile photos, added this session)

Every bucket in this project is created the same way (Dashboard → Storage → New bucket → toggle Public on) — or equivalently via SQL:

```
insert into storage.buckets (id, name, public)
values ('bucket-name', 'bucket-name', true)
on conflict (id) do nothing;
```

Public buckets mean anyone with the URL can view the file (fine for CMS marketing images) — but uploads still only happen through admin-authenticated dashboard forms, since the upload calls run client-side using the logged-in admin's session, not anonymous access.

RLS pattern used across this project

Two flavors, depending on whether the public homepage needs to read the table:

Public-read CMS content (hero_banners, about_section, faqs, testimonials, events, portfolio_projects, gallery_items, audio_tracks, audio_section, video_section, donate_section, site_settings, events_section):

```
alter table public.<table> enable row level security;

create policy "Public can read"
on public.<table> for select
using (true);

create policy "Admins can write"
on public.<table> for all
using ((auth.jwt() -> 'app_metadata' -> 'role') = 'admin')
with check ((auth.jwt() -> 'app_metadata' -> 'role') = 'admin');
```

Admin-only content (donations, contact_submissions):

```
alter table public.<table> enable row level security;

create policy "Admins can read"
  on public.<table> for select
  using ((auth.jwt() -> 'app_metadata' -> 'role') = 'admin');

-- contact_submissions also needs public INSERT (the contact form is
-- filled out by anonymous visitors), donations does not (only the
-- service-role webhook ever writes to it, which bypasses RLS entirely):
create policy "Anyone can submit"
  on public.contact_submissions for insert
  with check (true);
```

Important, hit this bug directly: table-level Postgres `GRANT` privileges are a *prerequisite* for RLS policies to even be evaluated. A table created via the raw SQL editor (not Supabase's Table Editor UI) does not automatically grant `authenticated` / `anon` the ability to touch it, even with correct RLS policies in place — this produces Postgres error `42501 permission denied for table X`. Fix with an explicit grant, e.g.:

```
grant select, update on public.profiles to authenticated;
```

Quick REST queries (curl) — works for any table above

```
SUPABASE_URL="https://your-project.supabase.co"
SERVICE_ROLE_KEY="..." # Project Settings → API → service_role (server-only, bypasses RLS)

# Read
curl -s "$SUPABASE_URL/rest/v1/<table>?select=*&order=created_at.desc&limit=10" \
  -H "apikey: $SERVICE_ROLE_KEY" -H "Authorization: Bearer $SERVICE_ROLE_KEY"

# Update a singleton section row
curl -s -X PATCH "$SUPABASE_URL/rest/v1/<table>?id=eq.1" \
  -H "apikey: $SERVICE_ROLE_KEY" -H "Authorization: Bearer $SERVICE_ROLE_KEY" \
  -H "Content-Type: application/json" -H "Prefer: return=representation" \
  -d '{"field": "value"}'

# Delete a row
curl -s -X DELETE "$SUPABASE_URL/rest/v1/<table>?id=eq.<row-id>" \
  -H "apikey: $SERVICE_ROLE_KEY" -H "Authorization: Bearer $SERVICE_ROLE_KEY"

# List storage buckets
curl -s "$SUPABASE_URL/storage/v1/bucket" \
  -H "apikey: $SERVICE_ROLE_KEY" -H "Authorization: Bearer $SERVICE_ROLE_KEY"

# Check whether the translations column exists on a table (non-destructive)
curl -s "$SUPABASE_URL/rest/v1/hero_banners?select=translations&limit=1" \
  -H "apikey: $SERVICE_ROLE_KEY" -H "Authorization: Bearer $SERVICE_ROLE_KEY"

# Inspect the FULL live schema (every table + column, straight from PostgREST)
curl -s "$SUPABASE_URL/rest/v1/" \
  -H "apikey: $SERVICE_ROLE_KEY" -H "Authorization: Bearer $SERVICE_ROLE_KEY"
```

Always use the **service_role** key for these — never the anon key, and never from browser-side code. It bypasses RLS entirely.

Stripe — the Donate (UPI QR) integration

A donor clicks **Donate Now** on the homepage. The browser calls our own API route, which asks Stripe to create a **PaymentIntent** for a fixed, admin-configured amount (`donate_section.default_amount`) using the **UPI** payment method with a QR display flow. Stripe returns a hosted QR image URL, which we show in a popup. When the donor actually pays (scanning with a real UPI app in live mode), Stripe fires a webhook to our server, which verifies it and writes a row into the `donations` table.

```
Browser → POST /api/donate/stripe-qr → Stripe PaymentIntents API → QR image URL Donor scans QR
(real money only in live mode) Stripe → POST /api/webhooks/stripe → verify signature → insert
into Supabase `donations` Admin → /admin/donate → SELECT * FROM donations
```

Get your API keys (Dashboard)

1. Stripe Dashboard → Developers → API keys
2. Toggle Test mode while developing (top-right switch)
3. Copy the Secret key → `STRIPE_SECRET_KEY` (`sk_test_...` for test, `sk_live_...` for real payments)

UPI eligibility: Stripe only supports UPI as a payment method for India-registered accounts with UPI explicitly enabled. Confirm this in Dashboard → Settings → Payment methods before going live.

Creating the UPI QR PaymentIntent

`app/api/donate/stripe-qr/route.ts` — the actual call that generates the QR:

```
const paymentIntent = await stripe.paymentIntents.create({
  amount: Math.round(amount * 100), // rupees → paise
  currency: "inr",
  payment_method_types: ["upi"],
  payment_method_data: {
    type: "upi",
    billing_details: {
      name: "Donor",
      address: {
        country: "IN",
        line1: "NA",
        city: "NA",
        state: "NA",
        postal_code: "110001",
      },
    },
  },
  payment_method_options: { upi: { flow: "qr" } },
  confirm: true,
});

// The QR image lives here – NOT under `upi_display_qr_code` as older
// docs/examples suggest. Confirmed empirically against the live API:
const imageUrl = paymentIntent.next_action
  ?.upi_handle_redirect_or_display_qr_code
  ?.qr_code
  ?.image_url_png;
```

Registering a webhook endpoint

Stripe won't send any events until an endpoint is registered for your current mode (test or live — they're separate). Two ways to do it:

Dashboard: Developers → Webhooks → Add endpoint → URL `https://yourdomain/api/webhooks/stripe`
→ event `payment_intent.succeeded` → copy the Signing secret.

Or via the API directly (what we used for this project):

```
curl -s https://api.stripe.com/v1/webhook_endpoints -u "$STRIPE_SECRET_KEY:" \
  -d "url=https://supabase-learning-animations.vercel.app/api/webhooks/stripe" \
  -d "enabled_events[]=payment_intent.succeeded"

# Response includes the signing secret, e.g.
# "secret": "whsec_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX"
# → this becomes STRIPE_WEBHOOK_SECRET
```

List existing endpoints (useful to check none are missing/duplicated):

```
curl -s https://api.stripe.com/v1/webhook_endpoints -u "$STRIPE_SECRET_KEY:"
```

Verifying the webhook

`app/api/webhooks/stripe/route.ts` — must read the raw body (not parsed JSON):

```
const rawBody = await request.text();
const signature = request.headers.get("stripe-signature");
const event = stripe.webhooks.constructEvent(rawBody, signature, webhookSecret);

if (event.type === "payment_intent.succeeded") {
  const paymentIntent = event.data.object;
  const supabase = createAdminClient();
  await supabase.from("donations").upsert({
    stripe_payment_intent_id: paymentIntent.id,
    amount: paymentIntent.amount / 100,
    currency: paymentIntent.currency.toUpperCase(),
    status: "captured",
    donor_email: paymentIntent.receipt_email ?? null,
  }, { onConflict: "stripe_payment_intent_id" });
}
```

Languages — English / Hindi / Punjabi (i18n)

The public site supports three languages via a switcher in the header. Two independent systems work together:

- **Static UI text** (nav labels, buttons, section headings, empty states — anything hardcoded in a component, not stored in Supabase) is translated using **next-intl** message files, hand/machine-translated once and shipped with the app.
- **CMS content** (hero titles, about text, testimonials, FAQs, events, etc. — anything an admin types into a dashboard form) is **auto-translated once when the admin saves**, using a free unofficial Google

Translate wrapper, and the result is cached in each row's `translations` column. Public page loads never call a translation API live — they just read whichever language column is already there.

Admin saves a form (e.g. Hero Banner) in English → row saves immediately, "Saved" toast fires right away → in the background: a Server Action translates the English fields to hi + pa → writes the result into that row's `translations` jsonb` column. Visitor picks a language in the header → sets a NEXT_LOCALE` cookie, calls router.refresh() → static text re-resolves via next-intl's message files → CMS content re-resolves via translations[locale] ?? original-english-field`

Translation engine: `google-translate-api-x` (npm) — a free, unofficial wrapper that calls the same endpoint `translate.google.com` itself uses. No API key, no billing account, no signup. Trade-off: it's unofficial, so it can occasionally be slow/rate-limited, and phrasing quality is "good enough," not perfect. This was a deliberate choice over Stripe-style paid/official APIs (e.g. Google Cloud Translation) specifically to keep the feature at zero ongoing cost.

Static UI text — next-intl

Packages: `next-intl` (^4.13.4). Configured in **"without i18n routing" mode** — no `[locale]` URL segment, since the admin panel and auth pages must stay English-only and this is largely a single-page site. The active locale is stored in a plain cookie, not the URL.

`lib/locales.ts` — the shared, side-effect-free constants (safe to import from client components):

```
export const LOCALES = ["en", "hi", "pa"] as const;
export type AppLocale = (typeof LOCALES)[number];
export const DEFAULT_LOCALE: AppLocale = "en";
export const LOCALE_COOKIE = "NEXT_LOCALE";

export function isAppLocale(value: string | undefined): value is AppLocale {
  return !!value && (LOCALES as readonly string[]).includes(value);
}
```

`i18n/request.ts` — next-intl's request config, reads the cookie server-side:

```
import { cookies } from "next/headers";
import { getRequestConfig } from "next-intl/server";
import { DEFAULT_LOCALE, isAppLocale, LOCALE_COOKIE } from "@lib/locales";

export default getRequestConfig(async () => {
  const cookieStore = await cookies();
  const cookieLocale = cookieStore.get(LOCALE_COOKIE)?.value;
  const locale = isAppLocale(cookieLocale) ? cookieLocale : DEFAULT_LOCALE;
  return {
    locale,
    messages: (await import(`../messages/${locale}.json`)).default,
  };
});
```

`next.config.ts` wraps the config with the next-intl plugin, and `app/layout.tsx` wraps `children` in `<NextIntlClientProvider>`, reading locale/messages via `getLocale()` / `getMessages()` from `next-intl/server`. Message files live at `messages/en.json`, `messages/hi.json`, `messages/pa.json`, namespaced per component (`Header`, `Footer`, `Portfolio`, `Faq`, `Contact`, etc.). Components use `useTranslations("Namespace")` (client components) or `getTranslations("Namespace")` (Server Components, e.g. the standalone `/portfolio` and `/portfolio/[id]` pages).

Bug hit and fixed: `i18n/request.ts` imports `next/headers`, which cannot be bundled into a client component. Originally the shared locale constants lived inside that same file; importing them from the header's client component broke the build ("You're importing a module that depends on next/headers... in the Pages Router"). Fixed by splitting the pure constants out into `lib/locales.ts` (no side-effectful imports) and having `i18n/request.ts` import *from* that file, not the other way around.

Language switcher

`app/actions/set-locale.ts` — a Server Action that just writes the cookie:

```
"use server";
import { cookies } from "next/headers";
import { isAppLocale, LOCALE_COOKIE } from "@lib/locales";

export async function setLocale(locale: string) {
  if (!isAppLocale(locale)) return;
  const cookieStore = await cookies();
  cookieStore.set(LOCALE_COOKIE, locale, { path: "/", maxAge: 60 * 60 * 24 * 365 });
}
```

The header's dropdown (desktop pill nav + mobile drawer) calls this action then `router.refresh()` — one call re-resolves both the static next-intl messages *and* re-fetches CMS rows with the new locale's translated fields, since both read the same cookie.

CMS content — auto-translate on save

`lib/translate.ts` — the core wrapper around `google-translate-api-x`. Batches every field for a row into **one HTTP request per locale** (not one request per field) by flattening fields/array-elements into a single object and calling the library's object-input batch mode with `rejectOnPartialFail: false`, so one bad field never blanks the whole translation:

```
export async function translateFields(
  fields: TranslatableFields,
  targetLocales: string[]
): Promise<TranslationsByLocale> {
  const flat = flattenFields(fields); // arrays become "features[0]", "features[1]", ...
  if (Object.keys(flat).length === 0) return {};

  const result: TranslationsByLocale = {};
  await Promise.all(targetLocales.map(async (to) => {
    try {
      const res = await translate(flat, { from: "en", to, rejectOnPartialFail: false });
      const translatedFlat = {};
      for (const key of Object.keys(flat)) translatedFlat[key] = res[key]?.text ?? flat[key];
      result[to] = unflattenFields(translatedFlat, fields); // rebuilds arrays in original order
    } catch (error) {
      console.error(`translateFields: translation to "${to}" failed`, error);
      // locale simply omitted from the result – never throws
    }
  }));
  return result;
}
```

`app/actions/translate-cms.ts` — one Server Action per table (10 total), each a thin wrapper: call `translateFields`, then update that row's `translations` column:

```

"use server";
const TARGET_LOCALES = ["hi", "pa"];

async function translateAndStore(table: string, id: string | number, fields: TranslatableFields)
{
  try {
    const translations = await translateFields(fields, TARGET_LOCALES);
    if (Object.keys(translations).length === 0) return;
    const supabase = await createClient();
    await supabase.from(table).update({ translations }).eq("id", id);
  } catch (error) {
    console.error(`translateAndStore: unexpected failure for ${table}#${id}`, error);
  }
}

export async function translateHeroBanner(id: string, fields: TranslatableFields) {
  await translateAndStore("hero_banners", id, fields);
}
// ...and translateAboutSection, translatePortfolioProject, translateTestimonial,
//   translateFaq, translateEvent, translateGalleryItem, translateAudioTrack,
//   translateDonateSection, translateVideoSection – same pattern, one per table.

```

Each of the 10 admin forms calls its matching action **fire-and-forget**, right after its own save already succeeded and toasted:

```

toast.success(isEditing ? "Banner updated." : "Banner added.");

translateHeroBanner(data.id, {
  badge_text: values.badgeText,
  title: values.title,
  description: values.description || null,
  primary_button_text: values.primaryButtonText || null,
  secondary_button_text: values.secondaryButtonText || null,
}).catch(() => {}); // never awaited – never blocks or delays the save toast

router.refresh();

```

Accepted trade-off: because translation is fire-and-forget, a row saved and then immediately viewed in Hindi/Punjabi within roughly a second may briefly still show English until the background write lands (usually 1–2 seconds). This was a deliberate choice over making the admin wait for translation before seeing "Saved" — the alternative (await the translate call before toasting) is a one-line change in each form if that trade-off ever needs to flip.

`lib/localize.ts` — the read-side counterpart, pure functions with no I/O:

```

export function localizeRow(row, locale, fields) {
  if (locale === "en" || !row.translations?.[locale]) return row;
  const localized = { ...row };
  for (const field of fields) {
    const value = row.translations[locale][field];
    if (value !== undefined) localized[field] = value;
  }
  return localized;
}
export function localizeRows(rows, locale, fields) {
  return rows.map((row) => localizeRow(row, locale, fields));
}

```

Called from every relevant `get*()` function in `lib/supabase/server.ts` (which reads the same locale cookie), and explicitly from `app/portfolio/[id]/page.tsx`, which bypasses `server.ts` with its own raw Supabase queries and has to remember to select `translations` and localize manually.

SQL — the migration that was actually run

Run once in the Supabase SQL editor (see the per-table `translations jsonb` column above for where it landed):

```

alter table hero_banners      add column if not exists translations jsonb not null default
'{}'::jsonb;
alter table about_section    add column if not exists translations jsonb not null default
'{}'::jsonb;
alter table portfolio_projects add column if not exists translations jsonb not null default
'{}'::jsonb;
alter table testimonials     add column if not exists translations jsonb not null default
'{}'::jsonb;
alter table faqs             add column if not exists translations jsonb not null default
'{}'::jsonb;
alter table events           add column if not exists translations jsonb not null default
'{}'::jsonb;
alter table gallery_items    add column if not exists translations jsonb not null default
'{}'::jsonb;
alter table audio_tracks     add column if not exists translations jsonb not null default
'{}'::jsonb;
alter table donate_section   add column if not exists translations jsonb not null default
'{}'::jsonb;
alter table video_section    add column if not exists translations jsonb not null default
'{}'::jsonb;

```

Shape of the column, e.g. for `hero_banners` :

```
{
  "hi": { "title": "...", "description": "...", "badge_text": "..." },
  "pa": { "title": "...", "description": "...", "badge_text": "..." }
}
```

Only free-text fields go in — never ids, links, dates, emoji, image URLs, or proper nouns (testimonial `name`, portfolio `client_name` / `category` are deliberately left untranslated).

Existing rows aren't retroactively translated. The pipeline only fires when an admin saves a form — content entered before this feature shipped stays English-only in Hindi/Punjabi until the admin opens and re-saves it once. There is no bulk backfill job; if one is ever needed, it would call the same `translateFields()` + column update logic in `lib/translate.ts` against every existing row in each of the 10 tables.

Regenerating the static message files

`scripts/translate-messages.mjs` (kept in the repo as a dev tool, not part of the running app) — reads `messages/en.json`, flattens it, and calls the same `google-translate-api-x` package in one batched request per locale to regenerate `messages/hi.json` and `messages/pa.json`:

```
node scripts/translate-messages.mjs
```

Bug hit and fixed: the translator initially mangled ICU placeholders — `"@ {year} CMS..."` came back with `{year}` itself translated into the target language (e.g. `{वर्ष}`), which breaks next-intl's variable substitution since the placeholder name no longer matches what the component passes at `t("copyright", { year })`. Fixed by masking every `{placeholder}` with a neutral token (e.g. `[[0]]`) before sending text to the translator, then restoring the original placeholder text afterward.

Adding a 4th language later

1. Add the locale code to `LOCALES` in `lib/locales.ts`.
 2. Add its label to `LOCALE_LABELS` / `LOCALE_SHORT` in `components/shadn/header.tsx`.
 3. Run `node scripts/translate-messages.mjs` to generate `messages/<code>.json` for static text (edit the script's `TARGETS` array first).
 4. Add `<code>` to `TARGET_LOCALES` in `app/actions/translate-cms.ts` so future CMS saves also translate into it.
 5. Existing CMS rows still need a manual re-save per item to pick up the new language, per the note above.
-

Environment variables

Variable	Where it's used	Where to get it
<code>NEXT_PUBLIC_SUPABASE_URL</code>	All Supabase clients	Supabase → Project Settings → API
<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	Browser-side client	Supabase → Project Settings → API
<code>SUPABASE_SERVICE_ROLE_KEY</code>	Webhook DB writes only	Supabase → Project Settings → API
<code>STRIPE_SECRET_KEY</code>	Creating PaymentIntents	Stripe → Developers → API keys
<code>STRIPE_WEBHOOK_SECRET</code>	Verifying webhook signatures	Stripe webhook endpoint's secret field
<code>EMAIL_USER</code> / <code>EMAIL_APP_PASSWORD</code>	Newsletter sending (nodemailer, Gmail SMTP)	Google Account → App Passwords (requires 2-Step Verification)
<code>NEXT_PUBLIC_SITE_URL</code>	Canonical links in outgoing emails	Your production URL, set manually

No new environment variables were needed for the language feature — `google-translate-api-x` requires no API key or billing account. Local vs. production are still separate: `.env.local` only affects `npm run dev` ; Vercel Production needs its own copies added via `vercel env add` .

Stripe CLI — local testing

```
# One-time login (opens browser to authorize the CLI)
stripe login

# Check which account/keys the CLI is using
stripe config --list

# Forward Stripe events to your local dev server. Prints a signing secret —
# use THIS one in local .env.local, not the Dashboard endpoint's secret.
stripe listen --forward-to localhost:3000/api/webhooks/stripe

# In a second terminal: simulate a successful payment without a real card/UPI app
stripe trigger payment_intent.succeeded

# Look up API/docs reference from the terminal
stripe docs search "upi payment method"
stripe docs api "POST /v1/payment_intents"
```

There's an npm script wired up for convenience: `npm run stripe` runs `stripe listen --forward-to localhost:3000/api/webhooks/stripe` .

Vercel — env vars & deploy

```
# See what's currently set on Production
vercel env ls production

# Add a var (pipe the value in so it's non-interactive)
printf '%s' 'sk_test_xxxxx' | vercel env add STRIPE_SECRET_KEY production

# Remove a stale one before replacing it
vercel env rm STRIPE_WEBHOOK_SECRET production --yes

# Pull production env vars down locally (alternative to manually copying)
vercel env pull --yes

# Deploy to production
vercel --prod --yes

# Check recent function logs (great for debugging a live 500)
vercel logs https://your-domain --json
```

Alias quirk on this project: every `vercel --prod` run re-attaches an old alias (`g-sap-animation-alpha.vercel.app`) automatically. After every deploy:

```
vercel alias remove g-sap-animation-alpha.vercel.app --yes
vercel alias set <newest-deployment-url> supabase-learning-animations.vercel.app
```

Gotchas we actually hit (and fixed)

Stripe / Donate:

- **Missing `billing_details.name`** : Stripe rejects a UPI PaymentIntent without it.
- **Missing `billing_details.address`** : the whole address object (not just `country`) is needed for UPI specifically, even though the generic schema lists every address field as "optional."
- **Wrong response field name:** the QR image is under `next_action.upi_handle_redirect_or_display_qr_code.qr_code.image_url_png` — not `upi_display_qr_code.image_data_url` as older examples suggest.
- **Minimum charge amount:** Stripe rejects INR amounts under roughly \$0.50 USD equivalent (`amount_too_small`). A ₹50 floor is enforced in the Zod schema, the admin form, and the API route.
- **No webhook endpoint registered:** easy to miss in production — always confirm with `curl https://api.stripe.com/v1/webhook_endpoints -u "$STRIPE_SECRET_KEY:"` .
- **Local `stripe listen` secret $\neq$ production webhook secret.** Two different signing secrets for two different registered destinations.

Languages:

- **Client/server import boundary:** importing anything from `i18n/request.ts` (which pulls in `next/headers`) into a client component breaks the build. Shared constants must live in a separate, side-effect-free module (`lib/locales.ts`).
- **ICU placeholders getting translated:** mask `{placeholder}` tokens before sending text through machine translation, restore them after — otherwise variable interpolation breaks.
- **Existing CMS content isn't retroactively translated** — only rows saved after the feature shipped get a populated `translations` column.
- **Testimonial names / portfolio client names** are deliberately excluded from translation — machine translation can mangle proper nouns.

Switching test mode → live mode (Stripe)

1. Stripe Dashboard → toggle off Test mode
2. Developers → API keys → copy the `sk_live_...` secret key
3. Developers → Webhooks → add a new endpoint (live-mode endpoints are separate from test-mode ones) → copy its `whsec_...`
4. Update `STRIPE_SECRET_KEY` and `STRIPE_WEBHOOK_SECRET` on Vercel Production with the live values
5. Redeploy

Do this deliberately, not by accident — live mode moves real money. Double-check the key prefix (`sk_live_` vs `sk_test_`) before deploying.

Generated for this project — the Supabase section reflects the live database schema (introspected directly, not recalled from memory), the Stripe section reflects the working, verified Donate integration, and the Languages section reflects the English/Hindi/Punjabi feature built and verified end-to-end in this session (translate-on-save confirmed against the live database, localized reads confirmed across all public routes in all three languages).